Create a Multiplayer Game

Project brief

Build a multiplayer game that works across devices, so people who aren't in the same room can play it together. Here are the requirements of the project:

- At least two players can join from separate devices
- The game has real rules a player could follow without you explaining them
- It's live at a URL or host other people can actually reach

This project can be completed in multiple ways:

- **Option A: Build on your own:** recommended for users with prior experience creating apps or sites with ChatGPT.
- **Option B: Step-by-step guide:** recommended for users new to creating apps or sites.

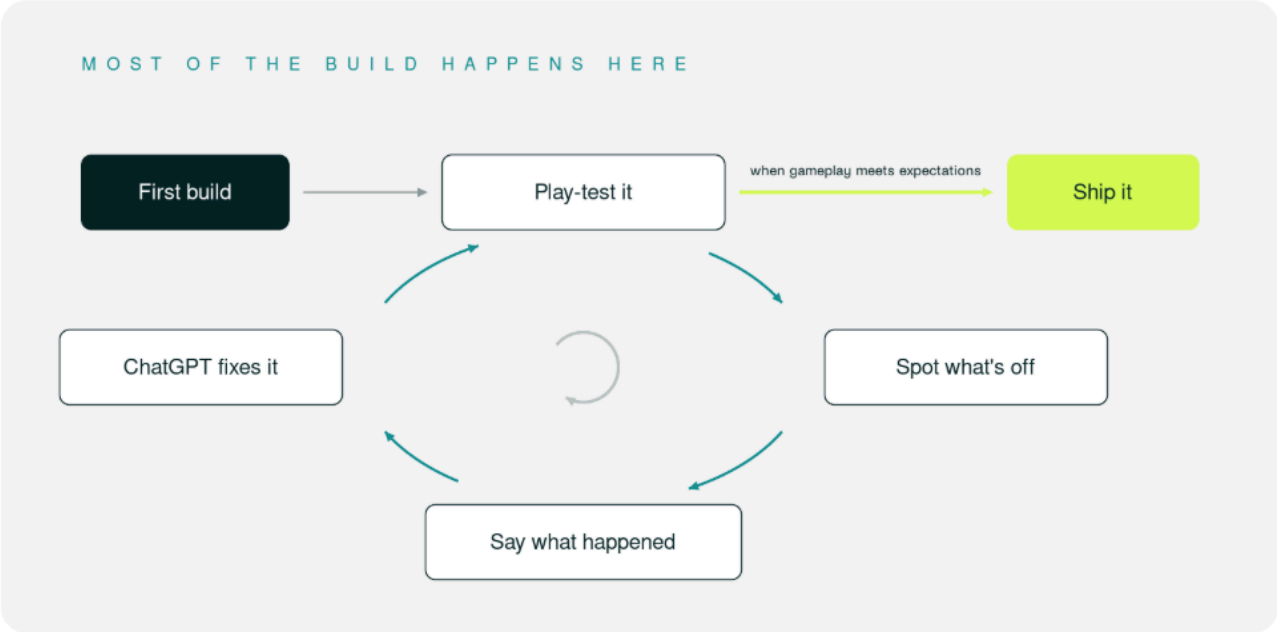
This mission requires **ChatGPT Work**. Here's how to access it:

- Use Work for free in the ChatGPT desktop app.
- **Have ChatGPT Plus?** Use Work in your browser or desktop app.
- **College Student?** If your school has ChatGPT EDU, log in with your school email to access ChatGPT Plus. If your school does not have ChatGPT EDU, you can get 4 months of ChatGPT Plus here.

Quick review: the two ideas this project runs on

Set it up to build, then let it run. You don't describe the game feature by feature. The starter prompt gives ChatGPT a role, states the requirements, and asks for a plan before it builds anything. You approve the plan, then ChatGPT starts to execute.

the code. This loop is usually where most of your build time goes, and it's a real skill. Be specific about what broke. Know when to push through a bug and when to back up and try another approach.



This loop is the same one practitioners run on real tasks every day, and it's often the part of building with AI that employers care about most. It's easy to assume the AI does the whole thing for you, but getting something to work is where the real thinking happens.

Option A: Build on your own

Recommended for users with prior experience creating apps or sites with ChatGPT.

Open ChatGPT

Create your game

Dive right in with ChatGPT and tell it to help you build your game. Make sure that the game is multiplayer and the rules are clear to any new player.

Publish and share

Once you've built and tested your project, make it live to share externally. Here are a few recommended options:

needed.

If you don't have access to Sites or want to use something else, here are other options. Just ask your chat **"give me step by step instructions on how to publish my project using [ ]"**

- **Netlify:** a free tool that puts your project online by uploading a folder. Good if you want the next easiest option, no coding tools required.
- **Vercel:** a free tool that puts your project online either by uploading a folder or connecting to GitHub. Good if you want fast performance and are open to a slightly more technical setup, or if a teammate wants to keep improving the project after you publish it.

Open the live URL in a private window to confirm it loads, then go to **Submit** and paste the link.

Option B: Step-by-Step Guide

Recommended for users new to creating apps or sites. Work through the steps in order and check each one off as you go to track your progress.

I. Open ChatGPT and design your game

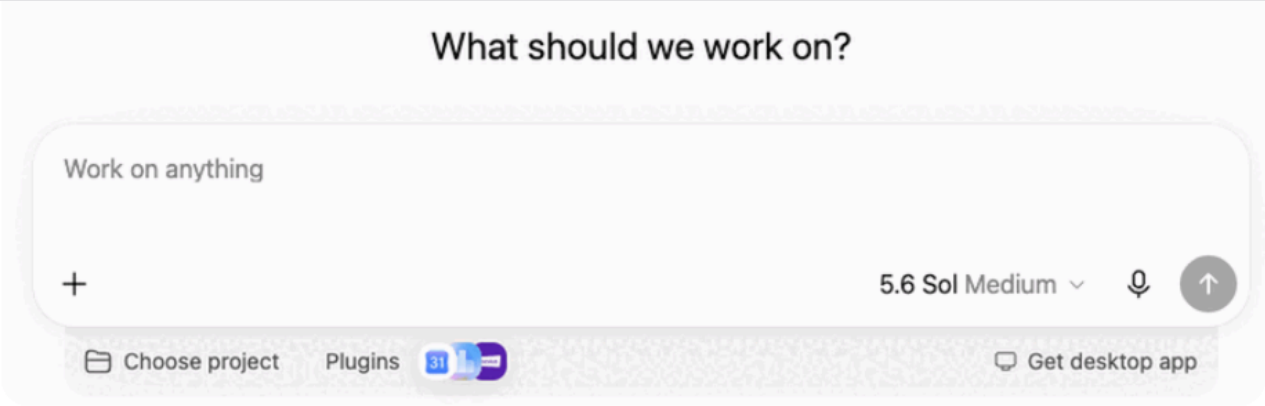
Open ChatGPT

- [ ] **Use the starter prompt to kick off the conversation**

👆 *note, you can check these off as you go to track your progress.*

Open ChatGPT and switch to **Work** at the top, then copy/paste in the prompt below. Work mode is what gives you the tools this build needs, so plain Chat won't get you there.

The prompt gives ChatGPT a role, lists what the result must include, and asks for a plan before building. That's a prompt pattern you can reuse any time you want AI to help you build something you couldn't alone.



Switch to Work at the top before you start. Chat mode doesn't have the tools this build needs.

```
Design and build a real-time party game for 2–8 players joining
by room code from their own device. Before building, talk me
through:
- The overall game concept. Ask me if I have any initial ideas
for favorite game types and work from there.
- Rules and scoring. How the gameplay happens, how it's scored,
and when it's over.

Summarize the rules before building anything, so I can approve.
```

Steer the conversation

Answer ChatGPT's questions as they come, and lean on it when you're unsure. Brainstorming out loud is the point! You don't need to keep any notes; the conversation holds every decision. When it's done, it'll play back a short spec of your final design.

1. Go one decision at a time. If a question stumps you, ask it to suggest a couple of options.
2. When it summarizes the design, read it closely. Fix anything that's off before you move on.
3. Give it the go-ahead only once the spec matches the game you want to play.

II. Build, test, and de-bug

Create the first version

Create a Multiplayer Game

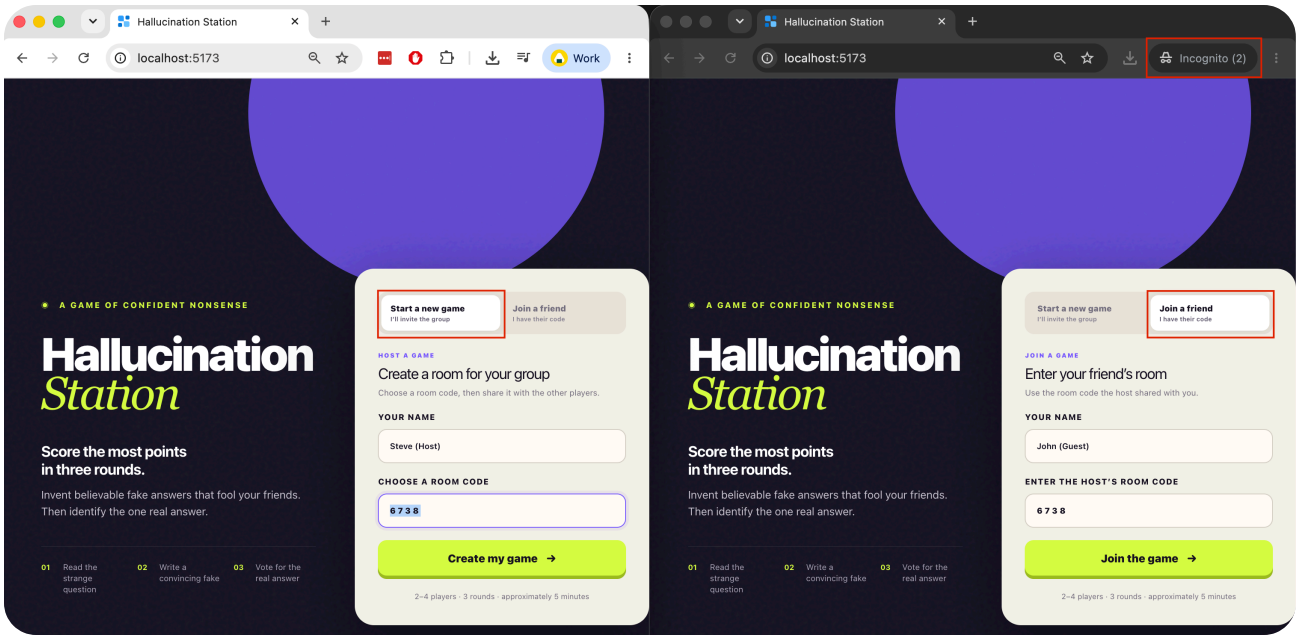
```
Go ahead and build the game we designed. Get the core loop
working first so that I can test it locally, then we'll polish.

Get the game running at a local address and give me the URL I
can paste into my browser to test.
```

The build will take a few minutes, and it'll give you an address that you can open in your browser. It'll look something like: *http://localhost:[a number]* - **you can copy/paste this URL directly into your browser just like any other URL.** The difference is that for now this is only running on your computer, not published on the internet.

Open and test

You now have a live game to play and test! Open two tabs in your browser - one normal and one Incognito. This will allow you to mimic having one user as a host and one as a joining player.



Left: creating a game room as a Host. Right: joining the room as a standard player.

Play a round or two and watch closely. Note anything that seems off: an update that doesn't show up, something appearing on a screen that shouldn't see it, an action that

De-bug

Debugging is a normal part of building, and it's worth getting comfortable with. Rather than fixing all the code yourself, you describe what's happening to ChatGPT in the same task and let it make the change. Bugs will come up, and it usually takes a few passes to work through them.

For each issue, tell it what you did, what you expected, and what happened instead. The more specific you are, the better the fix it can make:

```
When I did [what you did, and in which window], I expected [what
should have happened], but instead [what happened]. Can you fix
that? Keep the change small.
```

Then re-test in your two windows and repeat until the game runs smoothly.

III. Personalize the design

Set the layout, colors, and style

Now that the game works, customize the look and feel! Use the prompt below to make it yours:

```
Restyle the game and keep the game logic exactly as it is.
Mood: [e.g. late-night game show]
Colors: [two or three, plus one accent for buttons]
Layout: [e.g. one big question with the answers stacked below]
Apply it to every screen and keep the text easy to read on a
phone.
```

If you're stuck, ask ChatGPT for three very different directions and pick one.

IV. Make it live

Once you've built and tested your project, make it live to share externally. Here are a few recommended options:

Preferred

- **ChatGPT Sites:** if you've been working in ChatGPT, just ask it to **"publish this using @Sites"**. It'll check if you have access to **Sites** and publish it for you, no extra tools needed.

Second Option

If you don't have access to Sites (some workplaces or universities may restrict this), we recommend a simple tool called Netlify that will launch your site using a download of your project files.

1. Ask ChatGPT to get your game ready for Netlify

Once your game works locally, paste this into the same ChatGPT Work conversation:

```
Make this multiplayer game Netlify-ready for drag-and-drop
publishing. Preserve its current design and gameplay.

Replace any continuously running backend with a standard Netlify
Function at `netlify/functions/game.ts` and Netlify Blobs using
strong consistency and conditional writes.

The frontend must call `/.netlify/functions/game` directly. Do
not use `config.path` or an `/api/game` redirect. Keep helper
files outside `netlify/functions`.

Test a complete three-round game with two players, including
scoring and the final winner. Verify the production build and
packaged function.

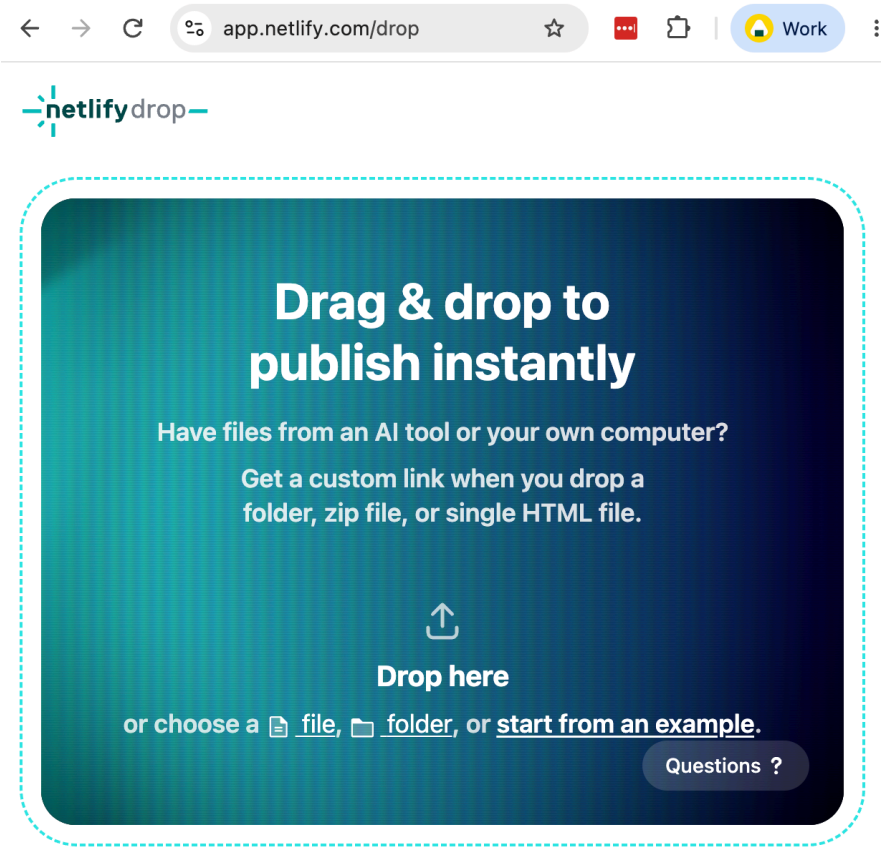
Package the complete project as `netlify-ready-game.zip`, ready
to upload directly to Netlify. Include root-level
`netlify.toml`, `package.json`, the lockfile, frontend source,
and Netlify files. Exclude `node_modules`, caches, and logs.

Do not require GitHub, Terminal commands, environment variables,
API keys, or manual database setup.
```

Wait for ChatGPT to finish. If it does not automatically download to your Desktop, click the filename in the chat to download it.

2. Add the files to Netlify Drop

A .zip folder is a compressed version of your files that makes sharing and storing easier. Go to https://app.netlify.com/drop and upload the .zip folder currently on your desktop.



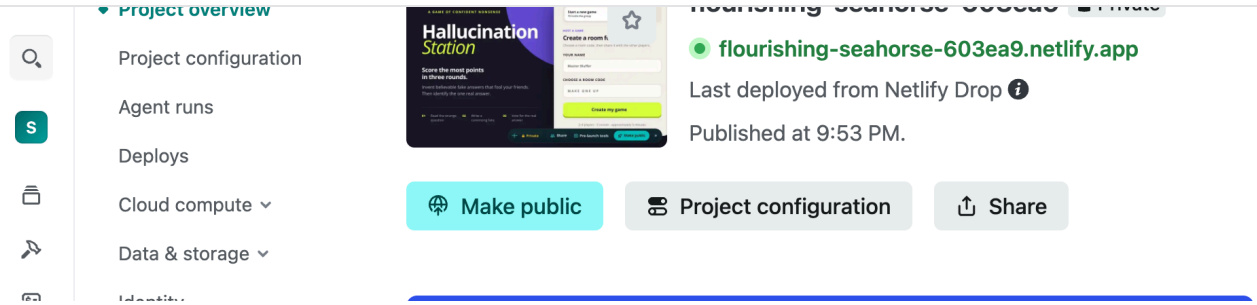
The Netlify upload interface. Upload your .zip file here.

If this is your first time on Netlify, it will ask you to create a free account.

3. Wait for Netlify to build your game

It will take a few minutes for Netlify to build your game. When it is done, you'll see a URL and the project status will turn green.

Create a Multiplayer Game



The Netlify interface after uploading files.

4. Test, make public, and share

Just as you did when the game was running on your own computer, open the URL that Netlify just created and test your game. Once you're confident the game can run, change the setting to public and share the link with friends.

Use the de-bugging skills you learned earlier. If the Netlify app hits an error, share the error message with ChatGPT and ask for guidance on what to fix.

Previous step

Next step